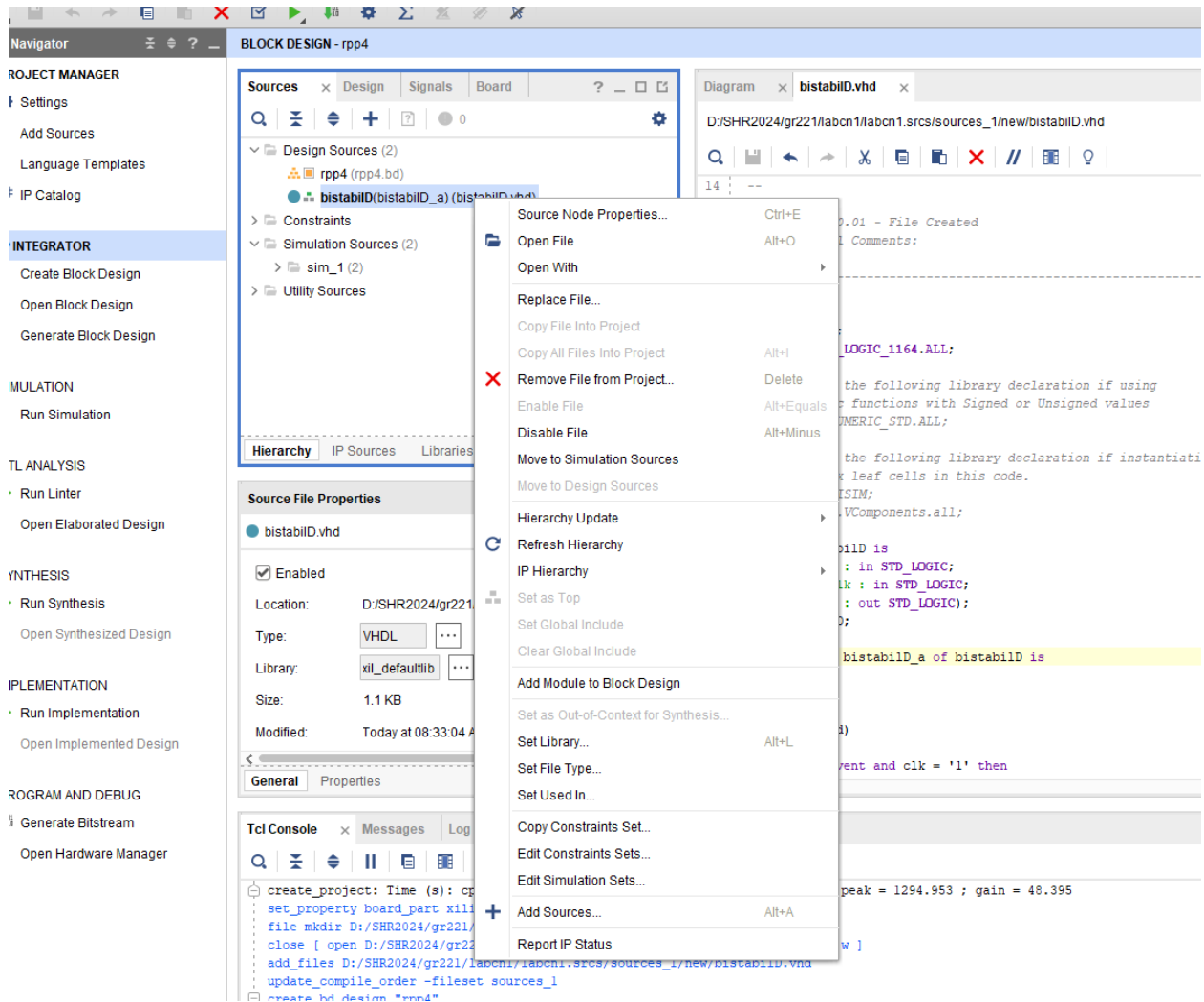


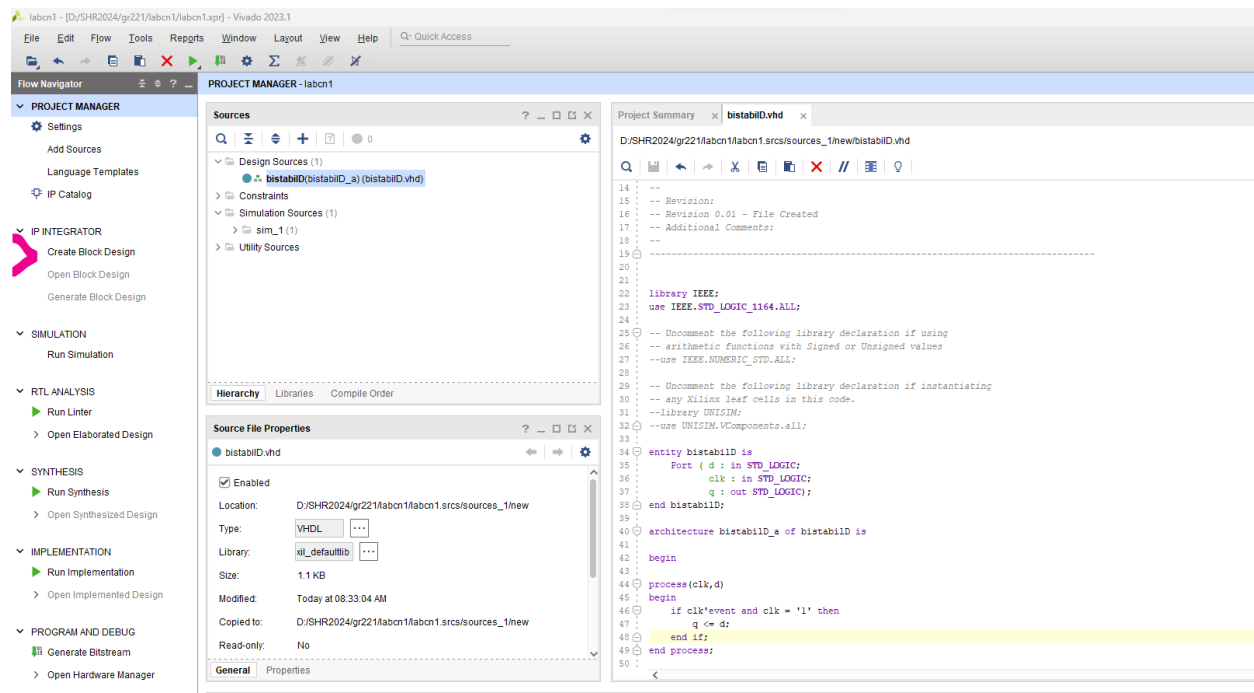
A. Adăugarea unei componente proprii (create în VHDL) la biblioteca Vivado

Din BlockDesign Sources se da click dreapta pe fișierul VHDL și se apasă Add module to block design

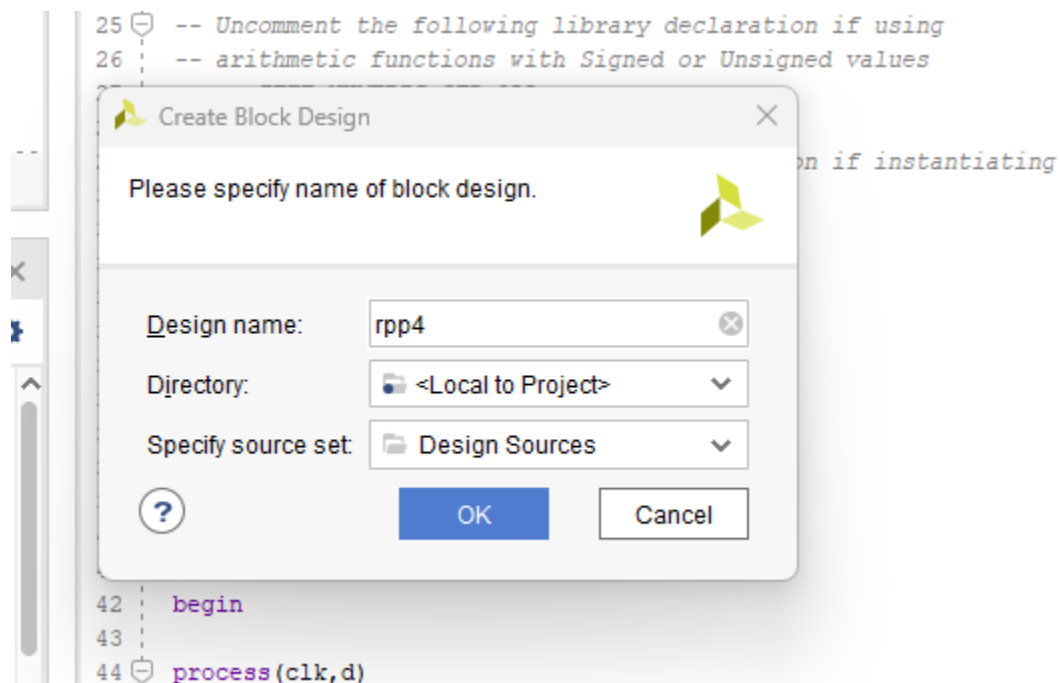


B. Lucru cu schematic în Vivado

Se selectează CreateBlock Design



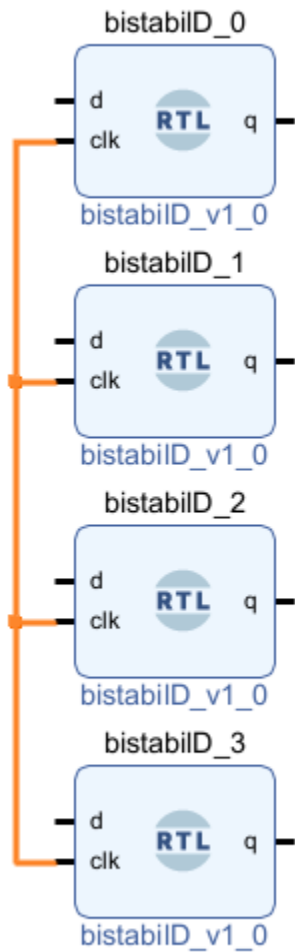
Se introduce numele fişierului



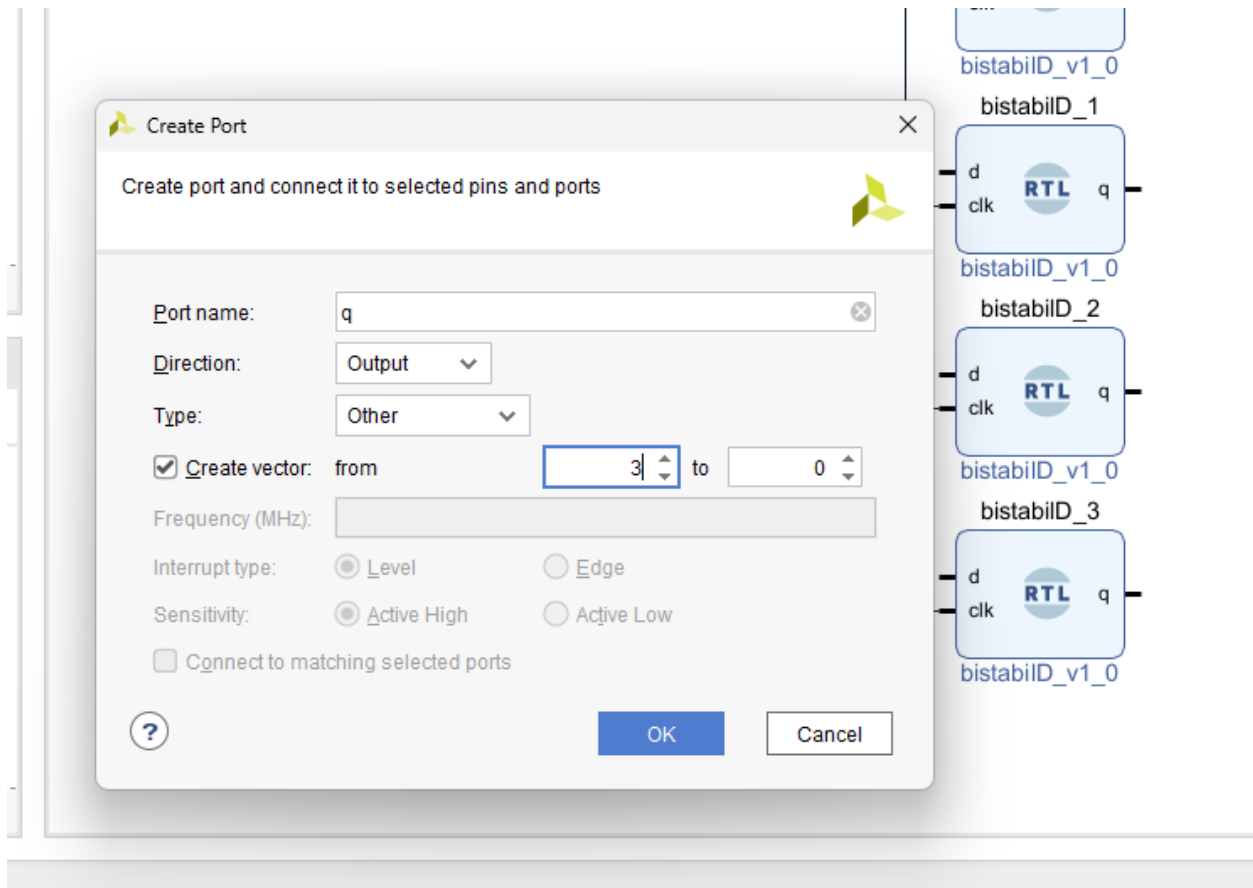
Se dă OK.

Adăugarea unei componente IP create (de exemplu bistabilD) se face ca la pasul A. Multiplicarea unei componente se poate face prin selectarea ei (click) și copy / paste.

Conexiunea între blocuri (module) se face prin click cu mouse-ul și drag unde se dorește efectuată conexiunea.

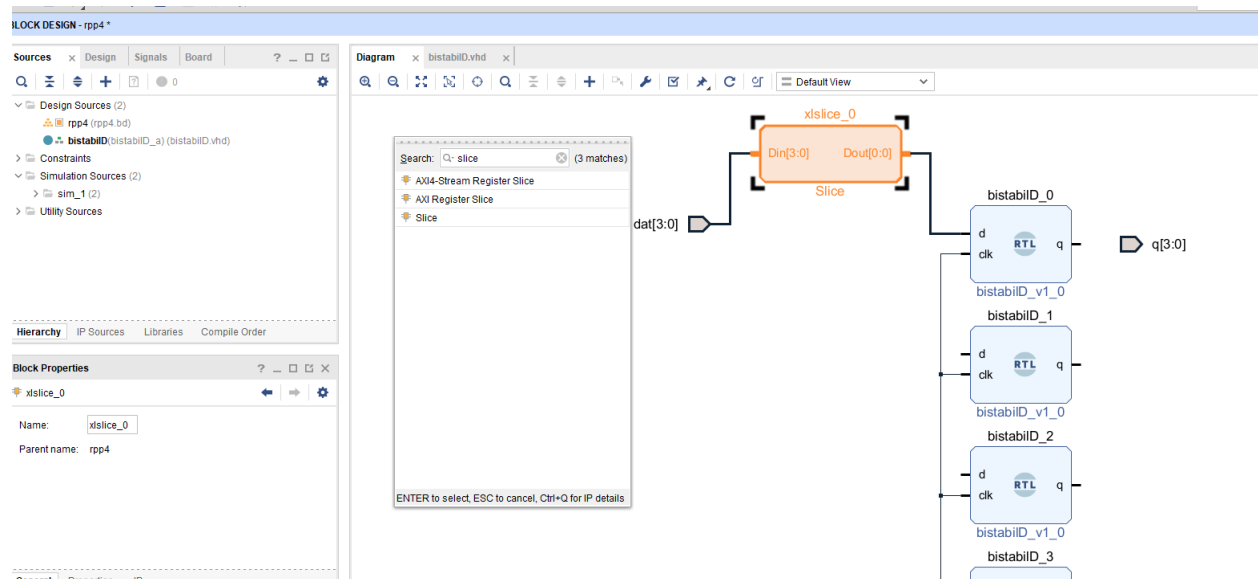


Plasarea porturilor (intrări și ieșiri) se face prin click dreapta și selectare Create port. Se scrie numele portului, direcționalitatea și dacă este cazul se precizează dacă este vector (magistrală).



Pentru conectarea unor biți (sau submagistrale) la o magistrală se procedează astfel:

- Se adaugă o componentă IP denumită Slice. Add IP (butonul plus), se scrie în căutare Slice și se dă OK.

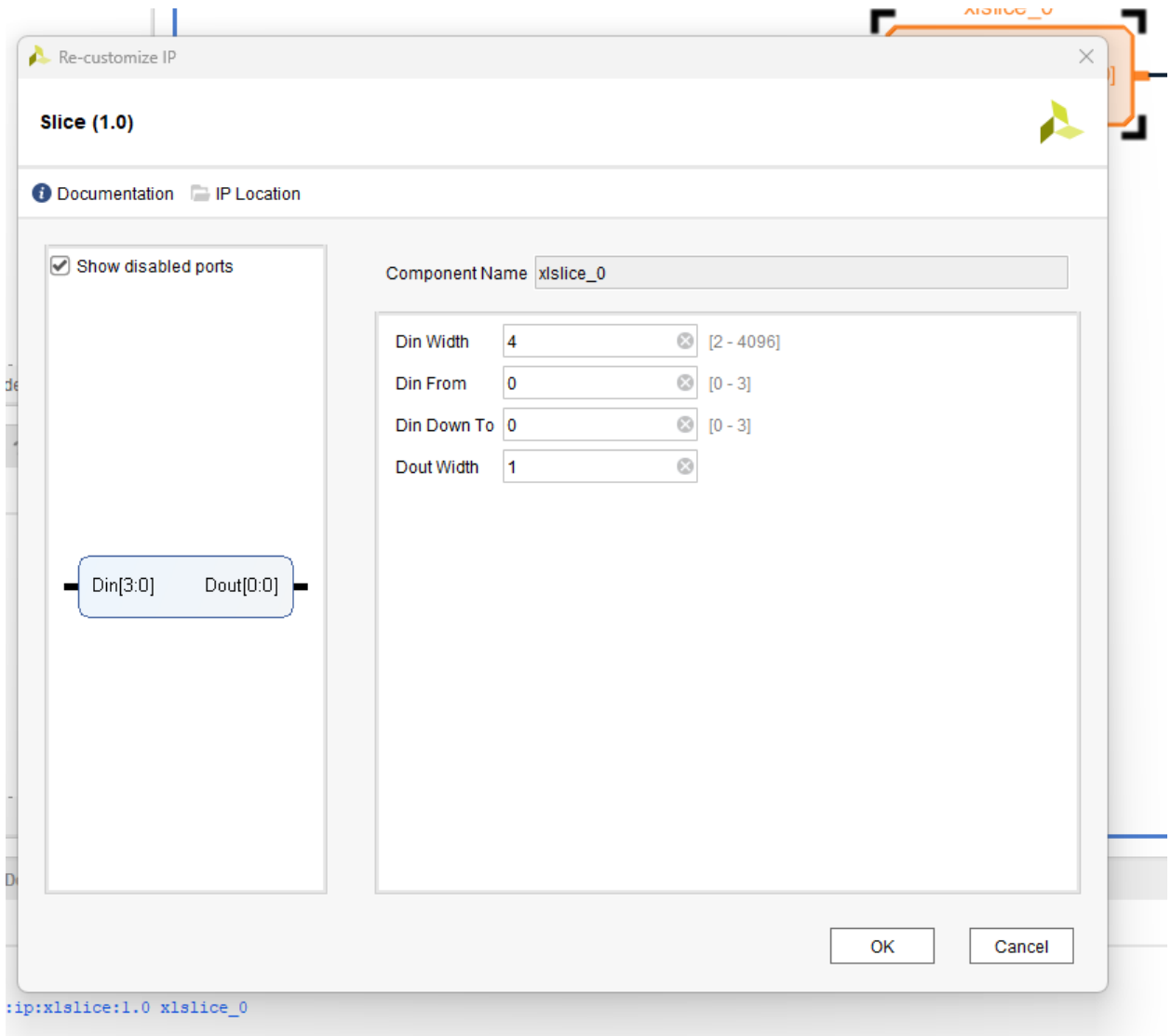


- Click dreapta pe componenta Slice respectivă și se selectează Customize Block

În Din Width se selectează dimensiunea magistralei de intrare

În Din From și Din DownTo se selectează segmentul din magistrala de intrare care se dorește să fie conectat la ieșire.

Pe Dout Width se selectează dimensiunea segmentului (magistralei) la ieșire.



O soluție alternativă pentru conectarea bit la magistrale sau segmente de magistrale la magistrale este editarea în VHDL a unui conector.

De exemplu, conectarea a patru intrări pe un bit fiecare, la o magistrală de ieșire pe 4 biți se face astfel:

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity conectorQ is
```

```
Port ( q0 : in STD_LOGIC;
```

```

    q1 : in STD_LOGIC;
    q2 : in STD_LOGIC;
    q3 : in STD_LOGIC;
    q : out STD_LOGIC_VECTOR (3 downto 0));
end conectorQ;

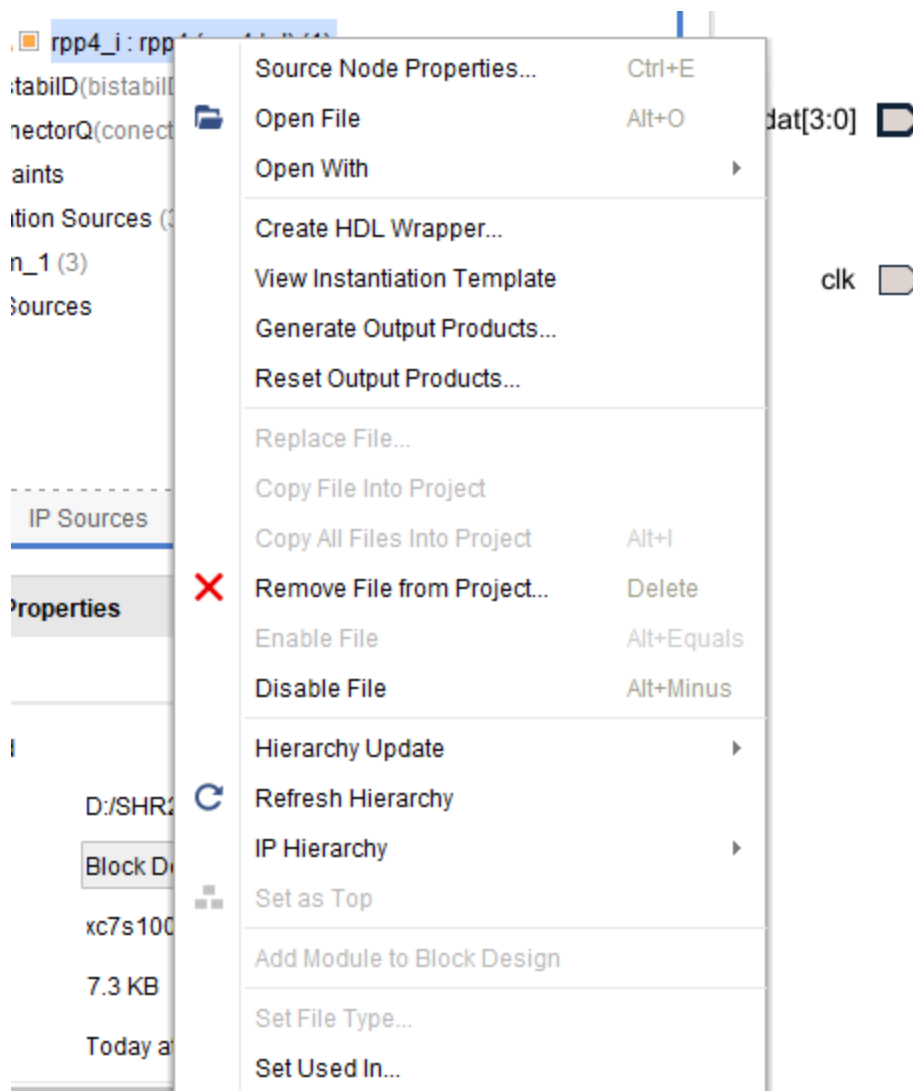
architecture conectorQ_a of conectorQ is
begin
    q(0) <= q0;
    q(1) <= q1;
    q(2) <= q2;
    q(3) <= q3;
end conectorQ_a;

```

Validarea schemei

După construirea schemei se dă click dreapta și Validate Design.

De asemenea, din meniul context care apare la click dreapta pe schemă în Design Sources dăm Create HDL Wrapper.



În acest moment este generat codul VHDL al schemei – practic toată schema este transformată într-un cod VHDL. După această etapă, schema ar trebui să fie setată în mod automat ca top level (apare îngroșată cu toate componentele subordonate).

Simularea în Vivado

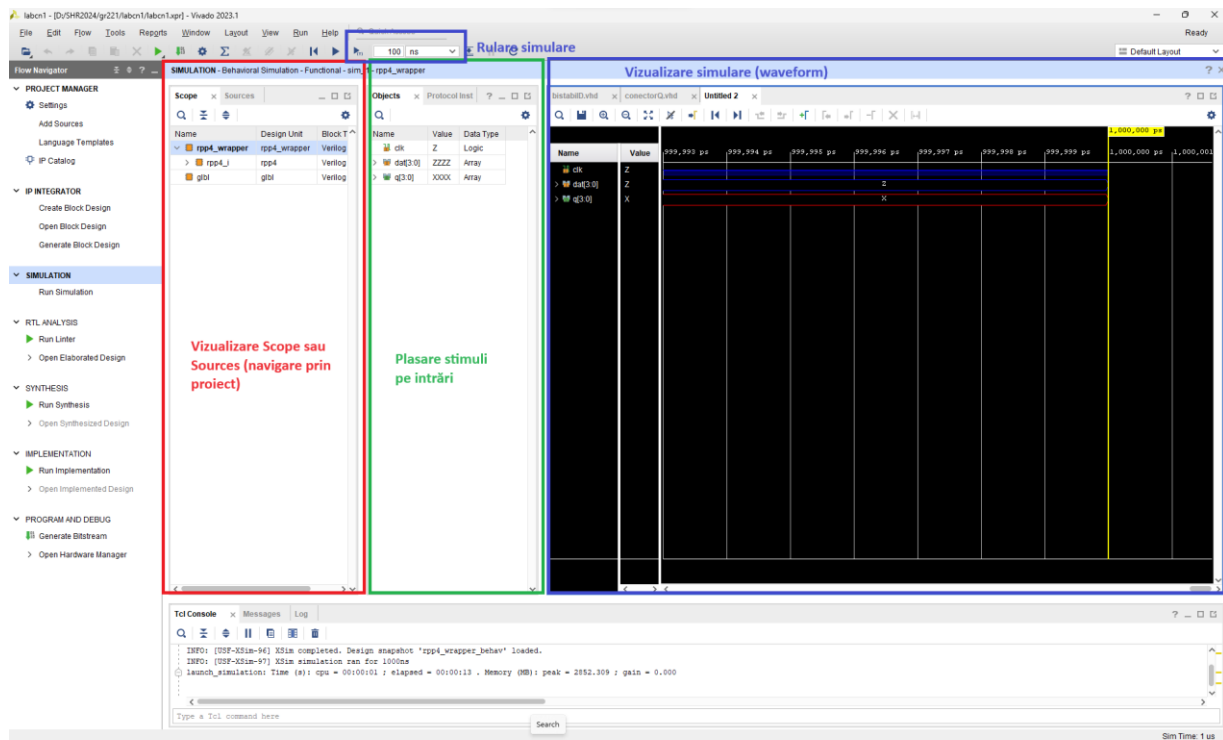
Simularea este o etapă intermediară care se efectuează opțional înainte de sinteză și implementare.

Pentru simulare, din Flow navigator, se rulează SIMULATION -> Run simulation

Atenție: Simularea se rulează pentru top level-ul curent. În cazul în care se dorește simularea pentru alte blocuri care nu sunt top level atunci acestea trebuie setate ca top-level (click dreapta pe el și Set as Top).

Din meniul care apare se dă Run Behavioral Simulation (singurul activ).

Fereastra principală devine Simulation. Ea este împărțită ca în figura de mai jos:

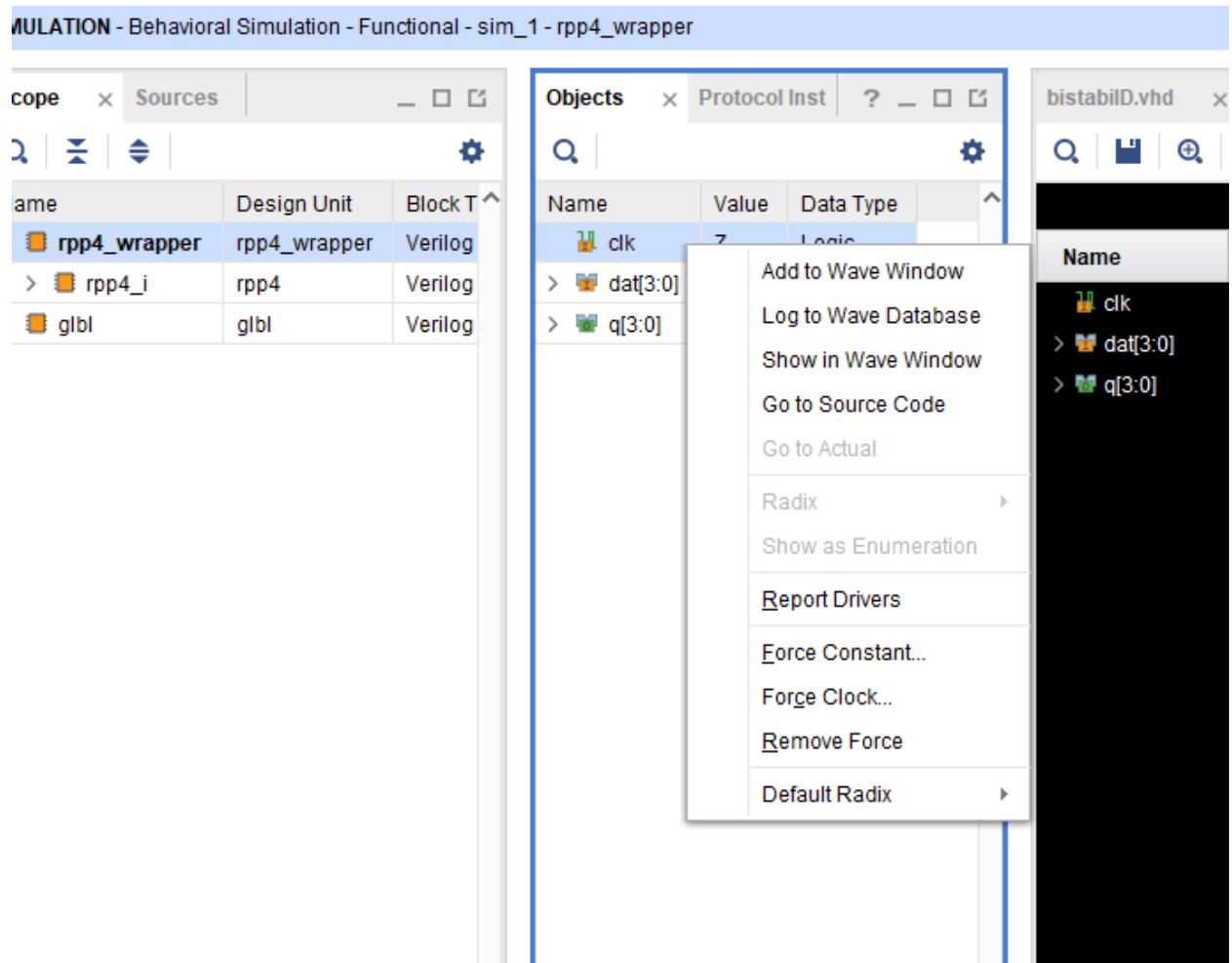


În Vizualizare Scope sau Sources se poate da pe tag-ul Sources și se poate naviga prin proiect. În plasare Stimuli sunt plasați stimulii pe intrările simulatorului în timp ce în vizualizare simulare se poate observa evoluția intrărilor și ieșirilor sub forma unor semnale. Rularea simulării se face de la butonul din zona rulare simulare (sus) pe intervalul de timp care este selectat. Se va rula întotdeauna simularea pentru un interval de timp (nu se va utiliza săgeata play ci săgeata play frame).

Întotdeauna, intrările (și opțional semnalele – dacă se dorește acest lucru) trebuie să aibă alocați stimuli pe ele – semnale simulate periodice sau aperiodice.

Pentru un stimul periodic se procedează astfel:

- În Plasare stimuli (vezi figura de mai sus) se dă click dreapta pe portul de intrare la care se dorește alocarea unui semnal stimul periodic. Din meniu se selectează Force clock:



În fereastra care apare, se vor selecta:

Value radix : baza de numerotație – de obicei este Binary sau Hexadecimal.

Leading edge: 1

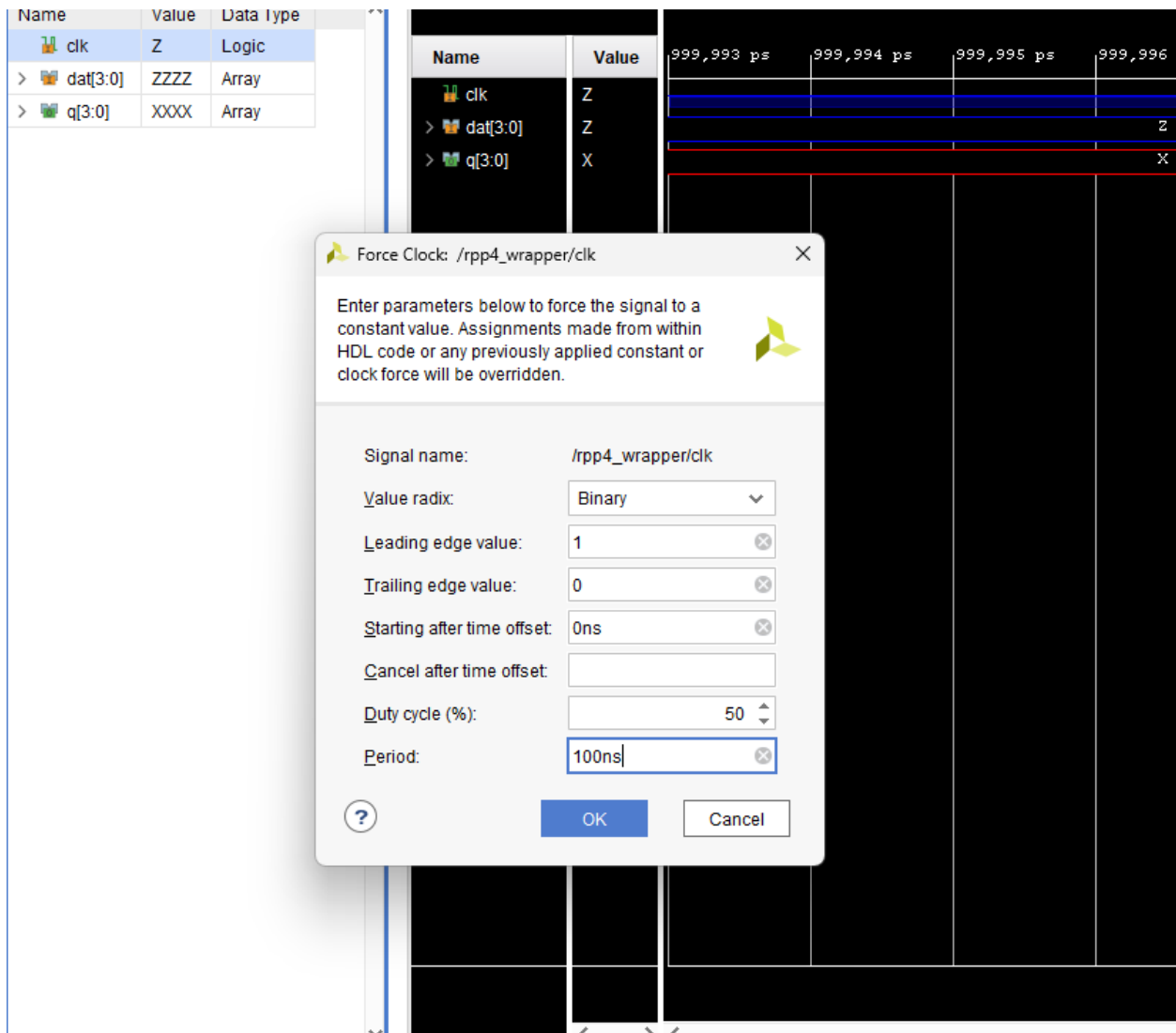
Trailing edge: 0

Starting after time offset: se introduce o valoare diferită de 0ns dacă se dorește defazarea semnalului periodic (cu o anumită întârziere).

Cancel after time offset: se introduce o valoare dacă se dorește ca semnalul periodic să fie întrerupt după un anumit timp.

Duty cycle: Factorul de umplere – se lasă de obicei 50%.

Period: Se introduce perioada semnalului.



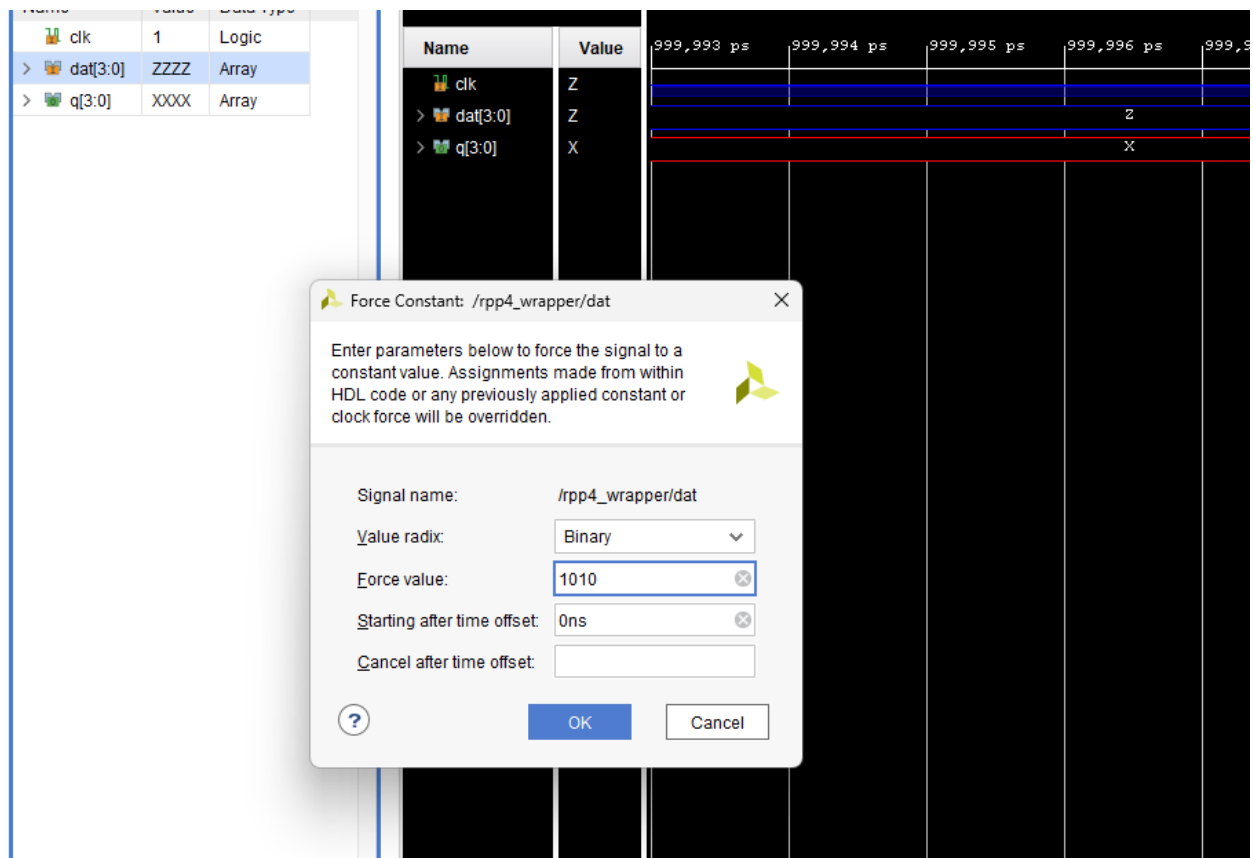
Pentru un stimul aperiodic (0 sau 1 logic pe un bit sau o valoare pe mai mulți biți pentru intrări de tip bus) se procedează astfel:

- Se dă click dreapta pe pe portul de intrare la care se dorește alocarea unui semnal stimul aperiodic. Se selectează Force Constant. În fereastra care apare vom selecta:

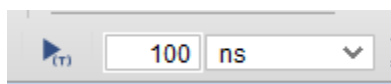
Value radix: de obicei este Binary dar pentru intrări de tip magistrală (pe mai mulți biți) poate fi și Hexadecimal.

Force value: se introduce valoarea care se dorește pe intrare. Dacă intrarea este pe un bit atunci poate fi 1 sau 0 iar dacă intrarea este pe mai mulți biți se introduce în formatul bcms bcmps (sau hexa de la digitul cel mai semnificativ la cel mai puțin semnificativ).

Starting after time offset se completează o valoare diferită de 0 ns dacă se dorește aplicarea unei întârzieri la aplicarea valorii. Cancel after time offset în cazul în care se dorește ca valoarea aplicată să se facă un anumit interval de timp apoi se revine la valoarea anterioară.



Rularea simulării se face prin apăsarea butonului play time frame și prin setarea unui interval de timp de rulare (sus sub meniu):



Poziționarea pentru vizualizarea rulării se face prin utilizarea scroll orizontal și butoane zoom.

